Arrowhead support systems in Go

Native implementation and interface catalogue

Abstract

This catalogue covers the fifteen active support systems in the local Java reference, including AIGym as a local extension. It records the delivered Go design, service interfaces, security boundaries and explicit remaining work. It is a development document; it does not assert full Java interchangeability or that any running cloud has changed.

> **Release boundary.** Do not replace a Java deployment solely because an HTTP route exists. Review each system's delimitations and the verification report; required advanced integrations and data migrations remain release gates.

Contents

1 AIGym **5**
1.1 System overview . . . 5
1.2 Produced and consumed services . . . 5
1.3 Important delimitations . . . 5
1.4 Security and persistence . . . 5
1.5 HTTPS interface operations . . . 6

2 Blacklist **7**
2.1 System overview . . . 7
2.2 Produced and consumed services . . . 7
2.3 Important delimitations . . . 7
2.4 Security and persistence . . . 7
2.5 HTTPS interface operations . . . 8

3 CertificateAuthority **9**
3.1 System overview . . . 9
3.2 Produced and consumed services . . . 9
3.3 Important delimitations . . . 9
3.4 Security and persistence . . . 9
3.5 HTTPS interface operations . . . 10

4 Choreographer **11**
4.1 System overview . . . 11
4.2 Produced and consumed services . . . 11
4.3 Important delimitations . . . 11
4.4 Security and persistence . . . 11
4.5 HTTPS interface operations . . . 12

5 Configuration **13**
5.1 System overview . . . 13
5.2 Produced and consumed services . . . 13
5.3 Important delimitations . . . 13
5.4 Security and persistence . . . 13
5.5 HTTPS interface operations . . . 14

6 DataManager **15**
6.1 System overview . . . 15
6.2 Produced and consumed services . . . 15
6.3 Important delimitations . . . 15

6.4 Security and persistence . . . 15
6.5 HTTPS interface operations . . . 16

7 DeviceQoSEvaluator 17
7.1 System overview . . . 17
7.2 Produced and consumed services . . . 17
7.3 Important delimitations . . . 17
7.4 Security and persistence . . . 17
7.5 HTTPS interface operations . . . 18

8 EventHandler 19
8.1 System overview . . . 19
8.2 Produced and consumed services . . . 19
8.3 Important delimitations . . . 19
8.4 Security and persistence . . . 19
8.5 HTTPS interface operations . . . 20

9 GateTunnel 21
9.1 System overview . . . 21
9.2 Produced and consumed services . . . 21
9.3 Important delimitations . . . 21
9.4 Security and persistence . . . 21
9.5 HTTPS interface operations . . . 22

10 Gatekeeper 23
10.1 System overview . . . 23
10.2 Produced and consumed services . . . 23
10.3 Important delimitations . . . 23
10.4 Security and persistence . . . 23
10.5 HTTPS interface operations . . . 24

11 Onboarding 25
11.1 System overview . . . 25
11.2 Produced and consumed services . . . 25
11.3 Important delimitations . . . 25
11.4 Security and persistence . . . 25
11.5 HTTPS interface operations . . . 26

12 PlantDescriptionEngine 27
12.1 System overview . . . 27
12.2 Produced and consumed services . . . 27

12.3 Important delimitations . . . 27
12.4 Security and persistence . . . 27
12.5 HTTPS interface operations . . . 28

13 SysMonitor **29**
13.1 System overview . . . 29
13.2 Produced and consumed services . . . 29
13.3 Important delimitations . . . 29
13.4 Security and persistence . . . 29
13.5 HTTPS interface operations . . . 30

14 TimeManager **31**
14.1 System overview . . . 31
14.2 Produced and consumed services . . . 31
14.3 Important delimitations . . . 31
14.4 Security and persistence . . . 31
14.5 HTTPS interface operations . . . 32

15 TranslationManager **33**
15.1 System overview . . . 33
15.2 Produced and consumed services . . . 33
15.3 Important delimitations . . . 33
15.4 Security and persistence . . . 33
15.5 HTTPS interface operations . . . 34

16 Verification and release gates **35**

1 AIGym

1.1 System overview

Local extension: nearest-neighbour model training and inference. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

1.2 Produced and consumed services

Produced HTTPS services: `aiGym`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

1.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Local extension, not an upstream mandatory Arrowhead 5.2.2 support system. Native k-nearest-neighbour behavior and persisted samples are implemented; all Java observation publishing semantics are not ported.

1.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

1.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
`GET`	`/health`	certificate
`POST`	`/aigym/samples`	identity
`POST`	`/aigym/predict`	identity
`GET`	`/aigym/observations`	identity
`GET`	`/aigym/status`	identity
`GET`	`/aigym/monitor/echo`	certificate
`GET`	`/aigym/general/mgmt/get-config`	management
`POST`	`/aigym/general/mgmt/logs`	management

2 Blacklist

2.1 System overview

Time-bounded system exclusion and discovery. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

2.2 Produced and consumed services

Produced HTTPS services: `blacklistDiscovery`, `blacklistManagement`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

2.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Time-bounded active/revoked records persist in JSON. Complete Java normalization, historical reactivation and query mode edge cases need golden-contract tests.

2.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

2.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
`GET`	`/blacklist/check/{systemName}`	identity
`GET`	`/blacklist/lookup`	identity
`POST`	`/blacklist/mgmt/create`	management
`DELETE`	`/blacklist/mgmt/remove`	management
`POST`	`/blacklist/mgmt/query`	management
`GET`	`/blacklist/monitor/echo`	certificate
`GET`	`/blacklist/general/mgmt/get-config`	management
`POST`	`/blacklist/general/mgmt/logs`	management

3 CertificateAuthority

3.1 System overview

Profile-constrained CSR issuance, revocation and trusted keys. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

3.2 Produced and consumed services

Produced HTTPS services: `certificateAuthorityManagement`, `certificateSigning`, `certificateValidation`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

3.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- CSR and CRL operations use a separately provisioned CA key; no CA key is generated/copied by this port. CRL export does not prove enforcement by every relying party. Java SQL import, automatic CRL file publication, HSM/OCSP/HA are absent.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

3.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

3.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
POST	/certificate-authority/sign	identity
POST	/certificate-authority/checkCertificate	identity
GET	/certificate-authority/crl	identity
GET	/certificate-authority/mgmt/certificates	management
DELETE	/certificate-authority/mgmt/certificates/{id}	identity
GET	/certificate-authority/mgmt/keys	management
DELETE	/certificate-authority/mgmt/keys/{id}	identity
POST	/certificate-authority/mgmt/revocations	management
PUT	/certificate-authority/mgmt/keys	management
POST	/certificate-authority/checkTrustedKey	identity
GET	/certificate-authority/monitor/echo	certificate
GET	/certificate-authority/general/mgmt/get-config	management
POST	/certificate-authority/general/mgmt/logs	management

4 Choreographer

4.1 System overview

Plan, executor and dependency-ordered session management. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

4.2 Produced and consumed services

Produced HTTPS services: `choreographerExecutor`, `choreographerManagement`, `choreographerSession`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

4.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Local synchronous orchestration and executor start/callback flow are implemented. Async orchestration callbacks return 501. Java executor dependency negotiation, optimal selection, session notifications and remote abort propagation are incomplete.

4.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

4.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/sessions	management
GET	/choreographer/mgmt/session-steps	management
GET	/choreographer/mgmt/worklog	management
GET	/choreographer/mgmt/executors	management
GET	/choreographer/mgmt/plans/{id}	management
DELETE	/choreographer/mgmt/plans/{id}	management
GET	/choreographer/mgmt/executors/{id}	management
DELETE	/choreographer/mgmt/executors/{id}	management
POST	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/plans/{id}/check	management
POST	/choreographer/mgmt/executors	management
POST	/choreographer/executor/register	identity
DELETE	/choreographer/executor/unregister	identity
POST	/choreographer/session/start	identity
POST	/choreographer/mgmt/sessions/start	management
DELETE	/choreographer/session/abort/{id}	identity
DELETE	/choreographer/mgmt/sessions/{id}	management
POST	/choreographer/executor/notify-step-done	identity
POST	/choreographer/orchestration-result/{correlationId}	identity
GET	/choreographer/monitor/echo	certificate
GET	/choreographer/general/mgmt/get-config	management
POST	/choreographer/general/mgmt/logs	management

5 Configuration

5.1 System overview

Versioned configuration storage and retrieval. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

5.2 Produced and consumed services

Produced HTTPS services: `configuration`, `configurationManagement`, `configurationProxy`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

5.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- JSON persistence replaces JPA/SQL. Proxy currently serves the local configuration store instead of Java external-provider fallback.

5.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

5.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/configuration/config/{systemName}	identity
GET	/configuration/proxy/config/{systemName}	identity
GET	/configuration/config/raw/{systemName}	identity
PUT	/configuration/mgmt/config/{systemName}	management
PUT	/configuration/proxy/save-config	identity
GET	/configuration/mgmt/config	management
DELETE	/configuration/mgmt/config/{systemName}	management
GET	/configuration/monitor/echo	certificate
GET	/configuration/general/mgmt/get-config	management
POST	/configuration/general/mgmt/logs	management

6 DataManager

6.1 System overview

SenML historian and authorized provider data proxy. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

6.2 Produced and consumed services

Produced HTTPS services: `dataHistorian`, `dataProxy`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

6.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Historian packs are persisted locally. Proxy resolves registered HTTPS get-data/put-data operations and uses its own identity. Complete SenML normalization, signalCount semantics, Java provider-configuration and MQTT ingestion are not ported.

6.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

6.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/datamanager/data-historian/systems	identity
GET	/datamanager/data-historian/systems/{systemName}/services	identity
GET	/datamanager/data-historian/systems/{systemName}/services/{serviceName}/data	identity
PUT	/datamanager/data-historian/systems/{systemName}/services/{serviceName}/data	identity
GET	/datamanager/data-proxy/systems	identity
GET	/datamanager/data-proxy/systems/{systemName}/services	identity
GET	/datamanager/data-proxy/systems/{systemName}/services/{serviceName}/data	identity
PUT	/datamanager/data-proxy/systems/{systemName}/services/{serviceName}/data	identity
GET	/datamanager/monitor/echo	certificate
GET	/datamanager/general/mgmt/get-config	management
POST	/datamanager/general/mgmt/logs	management

7 DeviceQoSEvaluator

7.1 System overview

Device measurement filtering and ranking. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

7.2 Produced and consumed services

Produced HTTPS services: `deviceQualityDataManagement`, `echo`, `generalManagement`, `qualityEvaluation`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

7.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Uses explicit DeviceMeasurementAgent input. The complete Java SNMP/OID discovery, augmentation and scaling engine is not ported; scores are weighted window means with missing-data rejection.

7.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

7.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
POST	/deviceqosevaluator/qualityevaluation/filter	identity
POST	/deviceqosevaluator/qualityevaluation/sort	identity
POST	/deviceqosevaluator/mgmt/query	management
POST	/deviceqosevaluator/mgmt/reload	management
GET	/deviceqosevaluator/monitor/echo	certificate
GET	/deviceqosevaluator/general/mgmt/get-config	management
POST	/deviceqosevaluator/general/mgmt/logs	management

8 EventHandler

8.1 System overview

Authorized subscriptions and event delivery. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

8.2 Produced and consumed services

Produced HTTPS services: `echo`, `eventHandlerManagement`, `eventPublish`, `eventSubscription`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

8.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- HTTPS callbacks use a persistent bounded outbox and at most three attempts. MQTT notifications, complete Java executor/backoff settings and distributed exactly-once guarantees are absent.

8.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

8.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
POST	/eventhandler/event-subscription/subscribe	identity
DELETE	/eventhandler/event-subscription/unsubscribe/{subscriptionId}	identity
GET	/eventhandler/management/subscriptions	management
PUT	/eventhandler/management/subscriptions/{subscriptionId}	management
DELETE	/eventhandler/management/subscriptions/{subscriptionId}	management
POST	/eventhandler/event-publish/publish	identity
GET	/eventhandler/monitor/echo	certificate
GET	/eventhandler/general/mgmt/get-config	management
POST	/eventhandler/general/mgmt/logs	management

9 GateTunnel

9.1 System overview

Authorized legacy-mTLS relay and tunnel sessions. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

9.2 Produced and consumed services

Produced HTTPS services: `echo`, `gateTunnelConnection`, `gateTunnelKey`, `gateTunnelManagement`, `generalManagement`. HTTPS service version 5.2.2. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

9.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Legacy-mTLS relay sessions are supported; control-session records are not a second AMQP tunnel data plane. AH-GT-1 encrypted requests are rejected. Consumer listeners are volatile and renegotiated after restart.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

9.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

9.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/gatetunnel/echo	identity
GET	/gatetunnel/publickey	identity
POST	/gatetunnel/connect_provider	identity
POST	/gatetunnel/connect_consumer	identity
POST	/gatetunnel/close_sessions	identity
GET	/gatetunnel/mgmt/sessions	management
POST	/gatetunnel/mgmt/sessions/close	management
POST	/gatetunnel/intercloud/provider	identity
POST	/gatetunnel/intercloud/consumer	identity
GET	/gatetunnel/monitor/echo	certificate
GET	/gatetunnel/general/mgmt/get-config	management
POST	/gatetunnel/general/mgmt/logs	management

10 Gatekeeper

10.1 System overview

Cross-cloud discovery and negotiation. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

10.2 Produced and consumed services

Produced HTTPS services: `echo`, `gatekeeperManagement`, `generalManagement`, `globalServiceDiscovery`, `interCloudNegotiation`. HTTPS service version 5.2.2. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

10.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Reuses the existing native Go legacy-mTLS relay engine, adapted to pinned clients. AH-GT-1 end-to-end encryption is absent. Broker ACLs and trusted neighbour provisioning remain required; no live federation is exercised by this delivery.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

10.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

10.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/gatekeeper/echo	identity
POST	/gatekeeper/init_gsd	identity
POST	/gatekeeper/init_multi_gsd	identity
POST	/gatekeeper/init_icn	identity
GET	/gatekeeper/mgmt/clouds	management
POST	/gatekeeper/mgmt/clouds	management
DELETE	/gatekeeper/mgmt/clouds/{operator}/{name}	management
GET	/gatekeeper/mgmt/relays	management
POST	/gatekeeper/mgmt/relays	management
GET	/gatekeeper/monitor/echo	certificate
GET	/gatekeeper/general/mgmt/get-config	management
POST	/gatekeeper/general/mgmt/logs	management

11 Onboarding

11.1 System overview

Controlled certificate, device and system onboarding. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

11.2 Produced and consumed services

Produced HTTPS services: `echo`, `generalManagement`, `onboarding`, `onboardingManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

11.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Enrollment records use JSON, not the Java database. System/identity/registry operations form a partial compensating saga, not a distributed transaction. Complete renewal, crash recovery, withdrawal of approved systems and prerequisite-service discovery require further work.

11.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

11.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/onboarding/echo	certificate
GET	/onboarding/mgmt/enrollments	management
GET	/onboarding/mgmt/enrollments/{deviceName}	management
PUT	/onboarding/mgmt/enrollments/{deviceName}	management
POST	/onboarding/certificate/name	enrollment
POST	/onboarding/certificate/csr	enrollment
POST	/onboarding/device	enrollment
POST	/onboarding/system	enrollment
POST	/onboarding/device/offboard	enrollment
POST	/onboarding/mgmt/enrollments/{deviceName}/offboard	management
DELETE	/onboarding/mgmt/enrollments/{deviceName}	management
GET	/onboarding/monitor/echo	certificate
GET	/onboarding/general/mgmt/get-config	management
POST	/onboarding/general/mgmt/logs	management

12 PlantDescriptionEngine

12.1 System overview

Plant topology and monitored alarm state. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

12.2 Produced and consumed services

Produced HTTPS services: `echo`, `generalManagement`, `plantDescriptionManagement`, `plantDescriptionMonitor`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

12.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Topology, include validation and alarm records are implemented. Monitoring views enrich explicitly named systems from SysMonitor. Full metadata resolution, periodic alarm reconciliation and orchestration-rule synchronization are incomplete.

12.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

12.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/plantdescriptionengine/mgmt/plant-descriptions	management
GET	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
POST	/plantdescriptionengine/mgmt/plant-descriptions	management
PUT	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
PATCH	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
DELETE	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
GET	/plantdescriptionengine/monitor/plant-descriptions	identity
GET	/plantdescriptionengine/monitor/plant-descriptions/{id}	identity
GET	/plantdescriptionengine/monitor/alarms	identity
GET	/plantdescriptionengine/monitor/alarms/{id}	identity
PATCH	/plantdescriptionengine/monitor/alarms/{id}	identity
GET	/plantdescriptionengine/monitor/echo	certificate
GET	/plantdescriptionengine/general/mgmt/get-config	management
POST	/plantdescriptionengine/general/mgmt/logs	management

13 SysMonitor

13.1 System overview

Monitoring subscriptions, history and durable observation forwarding. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

13.2 Produced and consumed services

Produced HTTPS services: `echo`, `generalManagement`, `sysMonitorHistory`, `sysMonitorManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

13.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- AMQPS intake and MQTT QoS1 outbox forwarding are implemented. MQTT uses Paho 3.1.1 rather than Java MQTTv5. Full live-broker/UI parity and imported Java history have not been certified. The default subscription is none until configured.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

13.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

13.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/sysmonitor/management/status	management
GET	/sysmonitor/management/providers	management
GET	/sysmonitor/management/subscriptions	management
PUT	/sysmonitor/management/subscriptions/{providerSystem}	management
DELETE	/sysmonitor/management/subscriptions/{providerSystem}	management
POST	/sysmonitor/management/reconcile	management
GET	/sysmonitor/history/systems	identity
GET	/sysmonitor/history/systems/{systemName}/latest	identity
GET	/sysmonitor/history/systems/{systemName}/observations	identity
GET	/sysmonitor/monitor/echo	certificate
GET	/sysmonitor/general/mgmt/get-config	management
POST	/sysmonitor/general/mgmt/logs	management

14 TimeManager

14.1 System overview

UTC reference time. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

14.2 Produced and consumed services

Produced HTTPS services: `echo`, `generalManagement`, `time`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

14.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- SNTP checks report offset/trust without changing the host clock. Full Java time-source caching, source-selection and trust policy are not ported.

14.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

14.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
`GET`	`/timemanager/time`	identity
`GET`	`/timemanager/monitor/echo`	certificate
`GET`	`/timemanager/general/mgmt/get-config`	management
`POST`	`/timemanager/general/mgmt/logs`	management

15 TranslationManager

15.1 System overview

Bridge discovery, negotiation and lifecycle. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

15.2 Produced and consumed services

Produced HTTPS services: `echo`, `generalManagement`, `translationBridge`, `translationBridgeManagement`, `translationReport`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

15.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Bridge lifecycle delegates to a configured InterfaceTranslator adapter over HTTPS. Java translator discovery, multi-stage data-model composition and translator-specific initialization are not ported; no translator means 503, never fabricated translation success.

15.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

15.5 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
`POST`	`/translation/bridge/discovery`	identity
`POST`	`/translation/bridge/negotiation`	identity
`POST`	`/translation/bridge/mgmt/discovery`	management
`POST`	`/translation/bridge/mgmt/negotiation`	management
`POST`	`/translation/bridge/mgmt/query`	management
`DELETE`	`/translation/bridge/abort/{bridgeId}`	identity
`DELETE`	`/translation/bridge/mgmt/abort`	management
`POST`	`/translation/event/report`	identity
`GET`	`/translation/monitor/echo`	certificate
`GET`	`/translation/general/mgmt/get-config`	management
`POST`	`/translation/general/mgmt/logs`	management

16 Verification and release gates

Build and static checks must cover all fifteen executables. Race-checked tests use isolated PKI and temporary state. Java-versus-Go golden contracts, real broker interoperability, database import and rollback, federation peers and LocalCloudManager readiness require separate evidence. No generated document labels these implementations production-ready.